

On-device LLM모델의 반응성 향상을 위한 동적 배치 최적화 기술*

정휘수⁰¹, 이지훈², 주성진², 이길호¹¹송실대학교 AI 융합학부²송실대학교 지능형반도체학과

gnltnwjstk@soongsil.ac.kr, ghoon00@soongsil.ac.kr,

cyse3062@soongsil.ac.kr, khlee.cs@ssu.ac.kr

Dynamic Batch Optimization for Responsive On-device LLM Inference

Whi-Soo Chung⁰¹, Seong-Jin Chou², Ji-Hoon Lee², Kil-Ho Lee¹¹School of AI Convergence, Soongsil University²Department of Intelligent Semiconductor, Soongsil University

요 약

임베디드 환경에서의 언어 모델 사용은 제한된 메모리 크기로 인해 메모리 문제가 빈번하게 발생하며 이는 추론 성능에 큰 영향을 미친다. 현존하는 배치 시스템은 추론 단계에서 사용 가능한 메모리 양과 처리해야 할 토큰 수를 고려하지 않고 고정된 배치 크기를 사용한다. 이러한 시스템은 불규칙한 길이의 입력들에 대해 높은 반응성을 보장할 수 없으며, Out-Of-Memory 문제를 초래한다. 본 논문은 이를 해결하기 위한 동적 배치 알고리즘을 제안한다. 알고리즘은 메모리 문제를 회피하면서 GPU 활용도를 극대화하기 위해 배치 구성을 최적화하여 온-디바이스 환경에서 언어 모델의 반응성을 개선했다.

1. 서론

최근 네트워크 지연 회피와 개인 정보 보호를 위해 언어 모델의 온-디바이스 추론 서비스가 급부상함에 따라, 높은 반응성이 요구되고 있다. 이를 위해 메모리 최적화 및 추론 가속 기술이 활발하게 연구되고 있다.

배치 처리는 여러 요청들을 병렬 처리함으로써 추론을 가속하는 대표적인 기법이다. 배치 처리는 사용자가 배치 크기를 어떻게 조정하는 지에 따라 GPU 활용도가 크게 달라질 수 있다. 추론 단계에서 사용되는 메모리의 크기는 배치 크기와 연산 처리할 토큰(Token) 수에 비례하기 때문에, 사용자는 시스템 메모리와 토큰 수를 고려해야만 최적의 배치 크기를 찾을 수 있다. 하지만 다양한 길이의 시퀀스들을 입력으로 받아, 처리할 토큰 수를 미리 알 수 없는 추론 서비스의 특성상 고정된 배치 크기 사용은 입력마다 GPU 활용도가 크게 변동하는 문제를 초래한다. 본 논문에서 제안하는 동적 배치 알고리즘은 입력 길이에 맞추어 배치 크기를 동적으로 조정하여, 높은 GPU 활용도를 통해 반응성을 높이는 것을 목표로 한다.

배치 연산에 필요한 메모리 사용량은 입력 길이 뿐만 아니라 출력 길이에도 영향을 받는다. 이전 연구[1]에서 정확한 메모리 사용량 측정을 위해 출력 길이 예측 모델을 사용하였지만, 온-디바이스 환경에서 예측을 위한 추가적인 추론 연산은 무시할 수 없는 오버헤드를 생성한다. 따라서 출력 길이에 대한 정보 없이 Out-Of-Memory (이하 OOM) 오류로부터 안전한 메모리 경계를 설정하는 것을 목표로 한다.

본 논문에서는 온-디바이스 환경에서 OOM 오류를 발생시키지 않으면서도 GPU 활용도를 최대화할 수 있는 동적 배치 최적화 알고리즘을 제시한다. 제시하는 알고리즘은 추론 연산의 메모리 사용량을 고려하여 배치를 구성할 뿐만 아니라, GPU 연산 최적화를 위해 각 배치내에 발생하게 될 패딩 연산을 최소화 하는 최적화를 수행한다. 이를 통해 OOM 회피를 보장함과 동시에 기존의 정적 배치 시스템 대비 최대 1.9배 감소한 추론 지연시간을 보였다.

2. 문제 설정

	Batch 1	Batch 2	Batch 4	Batch 8
Latency (sec)	43.18	36.81	33.36	OOM

표 1. 배치 크기 증가에 따른 latency 변화 양상
서론에서 언급한 대로, 정적인 배치 크기 설정은 GPU의 높은 활용도를 보장할 수 없다. 각 입력 시퀀스들이 배치

* 본 연구는 산업통상자원부 및 산업기술기획평가원의 자동차산업기술개발사업 (과제명: 자율주행차 전장부품 결함·오류 대응을 위한 기능 가변형 아키텍처 및 평가·검증기술 개발) (RS-2024-00404601) 및 과학기술정보통신부 및 정보통신기획평가원의 융합보안핵심인재양성사업 (RS-2024-00426853)의 연구결과로 수행되었음.

내에서 병렬로 처리될 때, 먼저 처리를 종료했음에도 다른 입력 시퀀스의 처리가 완료될 때까지 기다리는 대기(Wait) 시간이 생긴다. 배치 크기가 커질수록 배치 내에서 발생하는 총 대기 시간은 증가하고, 이는 병렬 처리로 인한 이득을 감소시킨다. 표 1은 NVIDIA Jetson AGX Orin 32GB[2]에서 배치 크기를 지수적으로 증가시키며 토큰의 수가 균등하게 분포된 입력 데이터들을 추론한 결과이다. 배치 2에서 4로 증가했을 때보다 배치 1에서 2로 증가했을 때의 지연시간 감소율이 훨씬 높은 수치를 보인다. 이는 대기 시간의 증가로 인해 배치 처리의 성능이 감소했기 때문이다. 또한 배치 8에서 관측된 OOM은 길이가 긴 입력 시퀀스를 처리할 때만 발생하였다. 이를 통해 적절한 배치 크기 설정이 비효율적인 배치 연산과 OOM 발생을 야기하는 것을 알 수 있다.

3. 시스템 설계

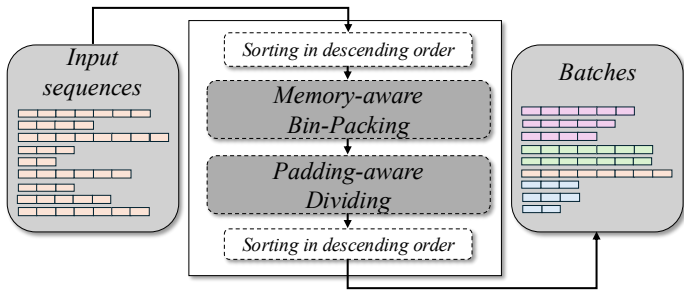


그림 1. 시스템 설계

앞서 언급한 바와 같이, OOM을 피하면서 높은 GPU 활용도를 확보하기 위해서는, 여유 메모리 크기와 배치 처리에 요구되는 메모리 크기를 고려하여 배치 크기를 동적으로 설정해야 한다. 그림 1은 본 논문에서 최적의 배치 조합을 찾아내는 과정을 단계적으로 보여준다. 알고리즘은 총 2단계로 구성되는데, 1단계에서 메모리-인지 bin패킹을 통해 각 토큰들을 적절하게 배치로 군집화하고, 2단계에서 각 배치에 포함되는 패딩 토큰에 따라 배치를 다시 분할하는 과정을 거친다. 두 과정을 거치면 유사한 길이의 토큰들이 배치로 군집화되어, GPU 연산의 효율성을 극대화할 수 있다.

3.1 Memory-aware Bin-Packing

대부분의 언어 모델은 트랜스포머 기반이다. 트랜스포머의 대표적인 특징인 셀프-어텐션(Self-Attention) 레이어들은 반복(iteration) 이후 출력한 하나의 토큰을 다음 반복의 입력으로 사용하는 자기 회귀적인 특성을 가진다. 이러한 과정에서 유발되는 불필요한 반복 연산을 줄이기 위해, 이전에 계산된 토큰의 키(Key)와 값(Value)을 캐시에 저장 및 재사용하는 방법을 사용한다. 하지만 KV 캐시는 상당한 메모리 공간을 차지한다. 배치 크기와 입력 및 출력의 토큰 수가 증가하면 KV 캐시에 필요한 메모리 양도 그만큼 늘어난다. 이에 따라 배치를 구성할 때에는, 토큰 처리에 필요한 KV캐시 크기를 고려하여, OOM 오류가 발생하지 않도록 배치 크기를 제한할 필요가 있다. 동적 배치 알고리즘은 토큰처리에 필요한 KV캐시 크기를

용량(Capacity)으로 하는 빈(Bin)을 채우는 빈-패킹(Bin-Packing) 문제를 해결하여 배치 조합을 구성한다. 이때 빈을 용량에 가깝게 가득 채울수록 병렬 작업으로 인한 GPU 활용도가 높아진다. 현존하는 배치 시스템은 입력과 출력 시퀀스 모두 패딩을 통해 배치 내에서 같은 길이를 유지하도록 한다. 이는 GPU의 고정 크기 블록 연산의 효율성을 높이는 반면에 패딩으로 인한 불필요한 연산과 메모리 낭비로 이어진다. 이에 따라 동적 배치 알고리즘은 배치 조합을 구성하는 과정에서 메모리를 제한하는 것 뿐만 아니라, 패딩 연산으로 인한 오버헤드를 최소화 하도록 조건을 추가한다. 따라서, 패딩을 최소화 할 수 있도록 빈-패킹 과정에서 FFD (First-Fit-Decreasing) 방식을 적용한다. 입력들을 토큰 수를 기준으로 내림차순 정렬하고, 정렬된 입력들에 대하여 First-Fit 빈-패킹으로 배치 조합을 구성한다. 이로 인해 비슷한 토큰 수를 가지는 입력들끼리 배치를 구성하도록 하여 토큰 길이 차이를 최소화한다.

3.2 Padding-aware Dividing

하지만 입력들의 길이 분포 불균형으로 인해 배치 내 입력들 간의 큰 토큰 길이 차이를 완전히 방지할 수는 없다. 이를 위해 동적 배치 알고리즘은 연산 처리될 총 토큰 수 대비 패딩의 비율을 추가적인 제약으로 적용하여 패딩으로 인한 오버헤드를 최소화한다. 그림 2에서 기존 배치 시스템의 경우 극단적인 길이 차이를 보이는 입력 시퀀스끼리 배치 조합을 구성하여 본래 필요한 연산보다 패딩으로 인한 연산을 더 많이 수행하게 된다. 이는 엄청난 성능 저하를 발생시킨다. 반면, 동적 배치 알고리즘은 빈을 충분히 채우지 않더라도 길이 차이가 큰 배치 조합을 회피하여 불필요한 연산을 최소화한다.

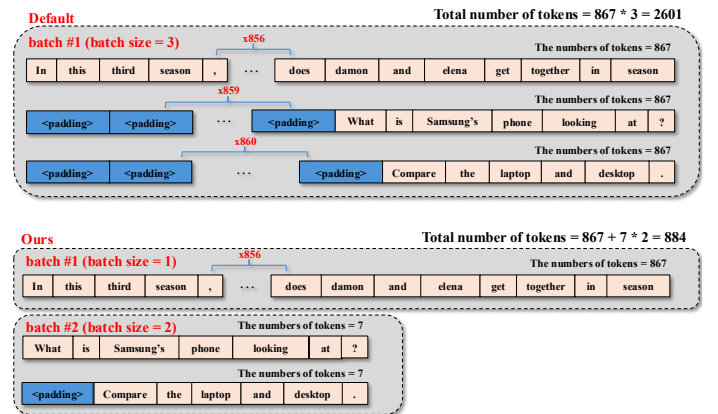


그림 2. 기존의 배치 구성과 동적 배치 알고리즘이 구성한 배치 조합 간의 패딩 차이

4. 동적 배치 알고리즘

그림 3의 동적 배치 알고리즘은 앞서 언급한 제약 조건들을 만족시켜 GPU 활용도와 메모리 사용을 최적화하도록 설계되었다. 먼저 입력 리스트 I 를 내림차순 정렬한 후, 입력 i_j 을 추가한 $Batch_{curr}$ 에 필요한 KV 캐시 크기 C_{kv} 를 계산한다. KV 캐시 크기는 $4 \times l_{batch}LH(l_{max} + k)$ [1]로

계산하며 l_{batch} 는 배치 크기, L 은 트랜스포머 레이어의 개수, H 는 숨겨진(hidden) 차원, l_{max} 는 배치 내 입력 길이의 최댓값, k 는 출력 길이의 최댓값을 의미한다. 앞서 언급한 대로 출력 길이 예측은 많은 오버헤드를 유발하기 때문에, 단답형이나 축약된 문장으로 제한하는 프롬프트를 사용하여 출력 길이 k 를 작은 값으로 대체할 수 있게 했다. 구해진 C_{kv} 는 이후에 시스템 메모리의 한계를 고려하여 설정된 T_m 을 초과하지 않도록 제한된다 (lines 12). 이후 전체 토큰 대비 패딩의 비율을 나타내는 $padding$ 을 계산한다. 해당 값은 패딩으로 인한 연산의 손해가 배치 크기를 늘림으로써 얻을 수 있는 이득을 넘지 않도록 T_q 를 초과하지 않게 제한한다 (lines 10-12). 모든 배치 조합을 구성하면, 각 배치 조합의 전체 토큰 수를 기준으로 내림차순 정렬한다. 이는 CUDA의 메모리 재사용 전략을 고려한 과정이다. CUDA는 메모리 footprint 감소를 위해 메모리 풀(Pool)을 사용하여 메모리를 재사용하려 한다. 메모리 풀의 블록들보다 큰 크기의 메모리 할당을 필요로 할 때, 남은 메모리가 부족하다고 판단되면 해제를 강행한다. 이때 해제하고 새로 할당하는 과정에서 비동기적으로 처리된 해제가 할당보다 지연되면, 동시에 여러 메모리 공간이 할당되어 메모리 제한을 초과하는 상황이 발생할 수 있다. 이러한 상황은 길이가 오름차순으로 정렬된 입력들이 순차적으로 추론될 때 종종 발생한다. 이를 방지하기 위해 배치 조합의 전체 토큰 수를 기준으로 내림차순 정렬하여 추론하면, CUDA의 메모리 재사용 기회를 높이는 동시에 OOM 오류를 회피할 수 있다.

Algorithm 1 Dynamic Batch Algorithm

```

1: Input:  $I$ 
2: Output:  $B_{sorted}$ 
3:  $I_{sorted} \leftarrow \text{SortDescending}(I)$ 
4:  $batch_{curr}, B \leftarrow \emptyset$ 
5:  $l_{max} \leftarrow 0$ 
6: for each  $i_j \in I_{sorted}$  do
7:    $l_{max} \leftarrow \max(l_{max}, len(i_j))$ 
8:    $l_{batch} \leftarrow |batch_{curr}| + 1$ 
9:    $C_{kv} \leftarrow 4 \times l_{batch} \times L \times H \times (l_{max} + k)$ 
10:   $total \leftarrow l_{max} \times l_{batch}$ 
11:   $padding \leftarrow total - \sum_{b_x \in batch_{curr}} len(b_x) - len(i_j)$ 
12:  if  $C_{kv} > T_m$  or  $\frac{padding}{total} > T_q$  then
13:     $B \leftarrow B \cup \{batch_{curr}\}$ 
14:     $batch_{curr} \leftarrow \emptyset$ 
15:    continue
16:  end if
17:   $batch_{curr} \leftarrow batch_{curr} \cup \{i_j\}$ 
18: end for
19: if  $|batch_{curr}| > 0$  then
20:    $B \leftarrow B \cup \{batch_{curr}\}$ 
21: end if
22:  $B_{sorted} \leftarrow \text{SortDescending}(B)$ 
23: return  $B_{sorted}$ 

```

그림 3. 동적 배치 알고리즘

5. 실험

5.1 실험 환경

실험은 Nvidia사의 Jetson AGX Orin 32GB 보드에서 진행했다. 모델은 Microsoft의 Phi3-medium-4k-instruct 모델을 사용했다. 해당 모델은 14B 파라미터를 가지며 채팅 형식의 Text-generation task를 처리할 수 있도록 훈련되었다. 실험은 AllenAI에서 제공한 OpenBookQA[5] 데이터셋을 가공하여 활용하였으며, 해당 데이터셋을 이루는

입력 시퀀스들은 토큰 수가 고르게 분포되어 있다. 실험과 구현에 사용한 프레임워크는 PyTorch (ver 2.3.0)[6]이다.

5.2 실험 결과

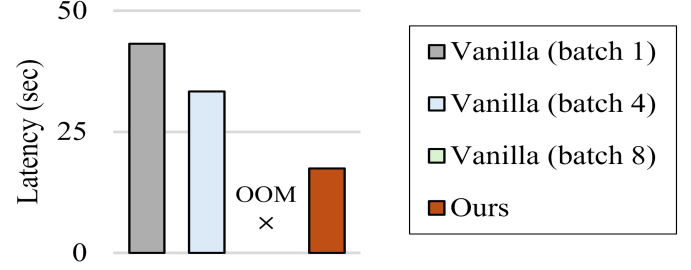


그림 4. 동적 배치 알고리즘을 적용한 결과

그림 4는 수정되지 않은 기존 PyTorch (이하 Vanilla)와 동적 배치 알고리즘을 적용한 PyTorch (이하 Ours)의 추론 결과를 비교한다. Vanilla의 경우, 배치 크기가 증가할수록 지연 시간이 감소하다가 배치 크기가 8일때 OOM을 발생시켰다. 이에 반하여 Ours는 OOM을 피하면서 Vanilla (batch 4) 대비 1.9배 빠른 지연 시간을 보였다. 이는 병렬 처리로 인한 성능 이득을 OOM이 발생하지 않는 지점까지 최대화하면서도 패딩 생성을 최소화했기 때문에, 기존 배치 시스템보다 안전성과 GPU 활용도에서 훨씬 효율적이다.

6. 결론

본 논문에서는 온-디바이스 환경에서 언어 모델의 추론 가속을 위한 동적 배치 알고리즘을 제안했다. 시스템 메모리와 배치 내에서 생성되는 패딩의 크기를 고려하면 OOM이 발생하지 않는 한에서 최대한 메모리를 활용할 수 있다. 또한 패딩의 비율을 제한함으로써 배치 처리의 효과를 상쇄시키는 오버헤드를 최소화할 수 있다.

참고 문헌

- [1] He, Yiyuan, et al. "UELLM: A Unified and Efficient Approach for LLM Inference Serving." *arXiv preprint arXiv:2409.14961* (2024).
- [2] Jetson AGX Orin for Next-Gen Robotics | NVIDIA [Online]. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>
- [3] microsoft/Phi-3-medium-4k-instruct [Online] <https://huggingface.co/microsoft/Phi-3-medium-4k-instruct>
- [4] Zhenmei Shi et al. Discovering the Gems in Early Layers: Accelerating Long-Context LLMs with 1000x Input Token Reduction. In arxiv, 2024
- [5] allenai/openbookqa Datasets at HuggingFace [Online] <https://huggingface.co/datasets/allenai/openbookqa>
- [6] PyTorch [Online]. <https://pytorch.org/blog/pytorch2-3/>